

diagnose_cross_validation_partitions.R

August 16, 2026

Contents

diagnose_cross_validation_partitions 1

Index 3

diagnose_cross_validation_partitions
Make Cross-Validation Partition Diagnostics

Description

Calculates full-model and fold-training counts from location assignments and aligned binary community responses for feasibility assessment.

Usage

```
diagnose_cross_validation_partitions(  
  data_locations = NULL,  
  data_assignments = NULL,  
  data_community_matrix = NULL,  
  cv_strategy = NULL,  
  min_taxon_locations = 1L,  
  min_taxon_samples = 1L,  
  include_full_model = TRUE  
)
```

Arguments

`data_locations` Location table returned by `[build_cross_validation_location_table()]`.

`data_assignments` Assignment table returned by a cross-validation assignment helper. May be empty when `'cv_strategy = "none"'` and only full-model diagnostics are required.

`data_community_matrix` Numeric binary matrix with one row per original sample row and one column per taxon. Row positions must match location `'row_indices'`.

`cv_strategy` Character scalar identifying the candidate strategy. Either `"spatially_stratified_group_kfold"`, `"leave_one_location_out"`, or `"none"` for full-model-only diagnostics.

`min_taxon_locations` Positive integer minimum number of training locations with a taxon presence.

`min_taxon_samples` Positive integer minimum number of training samples with a taxon presence.

`include_full_model` Logical scalar. If `'TRUE'`, prepend the full-model diagnostic row. Defaults to `'TRUE'`.

Details

Taxon viability is learned independently within each training partition. A retained binary taxon must meet both occurrence thresholds and contain at least one absence. MEM location counts equal the number of training locations; additional mode-specific MEM checks can raise the threshold in the feasibility assessor.

Value

Tibble matching `[resolve_cross_validation_strategy()]` input, with one optional full-model row and one row per repeat and fold.

Examples

```
data_locations <-
  tibble::tibble(
    location_id = c("a", "b"),
    n_samples = c(1L, 1L),
    row_indices = list(1L, 2L)
  )
data_assignments <-
  build_leave_one_location_out_assignments(data_locations)
diagnose_cross_validation_partitions(
  data_locations = data_locations,
  data_assignments = data_assignments,
  data_community_matrix = matrix(c(0, 1), ncol = 1L),
  cv_strategy = "leave_one_location_out"
)
```

Index

diagnose_cross_validation_partitions,
[1](#)